Verification of Robustness Transfer Between Neural Networks

Neta Nessing[1], Shir Drive[1], and Dana Drachsler Cohen[1]

Technion, Israel
{neta.nessing,shir.drive,ddana}@technion.ac.il

Abstract. Neural networks have achieved remarkable success across a wide range of applications, but they remain vulnerable to *adversarial examples*—maliciously perturbed inputs designed to deceive the network. Verifying the robustness of neural networks against such attacks is a challenging task. In particular, verifying the *global robustness* of a network—ensuring robustness for any input under a given perturbation function and scale—is computationally difficult. In this work, we address the problem of *robustness transfer verification*. Given a source network N_1 with a known robustness guarantee and a target network N_2, we compute a formal certificate guaranteeing N_2's global robustness property. We encode this problem as a mixed-integer program (MIP). Our approach supports seven perturbation types and employs four complementary tightening techniques: adversarial warm-starting, perturbation-difference interval propagation, inter-network interval bounds, and dependency analysis. We provide a complete formal analysis, including a proof that the transfer MIP yields a sound lower bound on the robustness improvement, a counterexample demonstrating that this bound can be strict, and a modified formulation that provides a matching upper bound over a restricted domain. Finally, we present a method for computing a sound upper bound during inference.

Keywords: Neural Network Verification · Global Robustness · Constrained Optimization

1 Introduction

Deep neural networks are successful in various tasks but are also susceptible to adversarial examples: malicious input perturbations designed to deceive the network [10,11,9]. Many adversarial attacks target image classifiers and compute either an imperceptible change, bounded by a small ε with respect to an L_p norm, or a perceivable change obtained by perturbing a semantic feature of the input, such as brightness, translation, or occlusion.

A popular safety property for understanding a network's resilience to such attacks is *local robustness*. Local robustness is parameterized by an input and its neighborhood: for a network classifier, the goal is to prove that the network classifies all inputs in the neighborhood the same. Several local robustness verifiers have been introduced for checking robustness in an ε-ball [6,8,7]. However,

local robustness is limited to reasoning about a single neighborhood at a time. To reason about a network's robustness over *all* possible inputs, VHAGaR [2] introduced the concept of *global robustness*: computing the *minimal globally robust bound*, the smallest confidence above which the network is provably robust to a given perturbation.

In practice, neural networks are rarely deployed in isolation. A common scenario involves training a more accurate network N_2 from an existing network N_1 via fine-tuning, distillation, or retraining. A natural question arises: *does the robustness guarantee of N_1 transfer to N_2?* More precisely, if N_1 is known to be globally robust above some confidence threshold $\delta_1(N_1)$, can we certify N_2's robustness without re-running the expensive global verification from scratch?

In this work, we address the problem of *robustness transfer verification*. Given a source network N_1 and a target network N_2, we compute a confidence margin δ_{diff} such that any input where N_2's confidence exceeds N_1's by more than δ_{diff} inherits N_1's robustness guarantee. Intuitively, δ_{diff} captures the worst-case confidence gap between the two networks in the regime where N_1 is certifiably robust but N_2 can still be fooled. This problem is challenging for two main reasons. First, it requires encoding three forward passes — $N_1(x)$, $N_2(x)$, and $N_2(x')$ — simultaneously in a single optimization problem. Second, the perturbation introduces complex dependencies between the clean and perturbed computations that must be captured precisely for the resulting bounds to be tight.

We extend VHAGaR with a *transfer mode* that encodes the robustness transfer problem as a mixed-integer program (MIP). Our MIP simultaneously models three forward passes — $N_1(x)$, $N_2(x)$, and $N_2(x')$ — linked by the perturbation constraint $x' \in \mathcal{I}_\varepsilon(x)$. To make the resulting MIP tractable, we introduce four complementary tightening techniques: (1) a *hyper-adversarial attack* that provides a warm-start solution via projected gradient descent, (2) *perturbation-difference interval propagation* that bounds the activation differences between clean and perturbed passes, (3) *inter-network interval bounds* exploiting the structural similarity between N_1 and N_2, and (4) *dependency analysis* that identifies monotone relationships between activations. Our approach currently supports the L_∞ perturbation and six semantic feature perturbations.

We provide a complete formal analysis of the transfer MIP's guarantees. We prove that the computed margin δ_{diff} provides a *sound lower bound*: $\delta_1(N_2) \geq \delta_1(N_1) + \delta_{\text{diff}}$, meaning N_2's global robustness bound is at least the sum of N_1's bound and the transfer margin. We then exhibit a concrete counterexample with two simple linear networks demonstrating that the reverse inequality is *false* in general — the bound is strict. Finally, we propose a modified transfer MIP, replacing the constraint that N_1 is robust with the constraint that N_1 is *foolable*, and prove that the resulting quantity $\hat{\delta}_{\text{diff}}$ satisfies the complementary upper bound $\delta_1(N_1) + \hat{\delta}_{\text{diff}} \geq \hat{\delta}_1(N_2)$.

In summary, our main contributions are: (1) formalizing the robustness transfer problem and encoding it as a MIP with four tightening techniques (Section 5), (2) proving that the transfer MIP yields a sound lower bound on N_2's robustness improvement over N_1 (Section 7), (3) exhibiting an explicit counterexample

showing the bound is strict, with a precise characterization of the gap (Section 7), and (4) designing a modified formulation that provides a matching upper bound over the joint-foolable domain (Section 7).

The rest of this paper is organized as follows. Section 2 provides background on network classifiers and formalizes the central quantities. Section 3 defines the supported perturbation models. Sections 4 and 5 present the standard-mode and transfer-mode MIP formulations, respectively. Section 6 discusses the two attack constraint variants. Section 7 provides the full formal analysis. Section 8 describes the four tightening techniques. Section 9 gives the complete MIP formulation, and Sections 10 and 11 present the runtime inference procedure and conclusions.

2 Preliminaries

In this section, we provide background on neural network classifiers and formalize the central quantities used throughout this paper. We begin with the definitions of networks and confidence margins, then introduce the notions of robustness and foolability that underlie both the standard-mode and transfer-mode formulations.

2.1 Neural Network Classifiers

A classifier maps an input into a score vector over a set of labels (also called classes) C, i.e., it is a function $N : [0,1]^n \to \mathbb{R}^{|C|}$. Given an input x, its classification is the label with the maximal score: $c = \text{argmax}(N(x))$. We focus on classifiers implemented as neural networks. A neural network consists of an input layer followed by L layers, where the last layer is the output layer. Layers consist of neurons, each connected to some or all neurons in the next layer.

We denote by $z_{m,k}$ the k-th neuron in layer m, for $m \in \{0, \ldots, L\}$ (where 0 denotes the input layer) and $k \in \{1, \ldots, k_m\}$. The input layer z_0 passes the input x to the next layer (i.e., $z_{0,k} = x_k$ for all $k \in [n]$). In a fully-connected layer, a neuron $z_{m,k}$ gets as input the outputs of all neurons in the preceding layer. It has a weight $w_{m,k,k'}$ for each input and a single bias $b_{m,k}$. Its function is the weighted sum of its inputs and bias followed by an activation function. Formally, the function of a non-input neuron is $z_{m,k} = \text{ReLU}(\hat{z}_{m,k}) = \max(0, \hat{z}_{m,k})$, where $\hat{z}_{m,k}$ is the weighted sum: $\hat{z}_{m,k} = b_{m,k} + \sum_{k'=1}^{k_{m-1}} w_{m,k,k'} \cdot z_{m-1,k'}$. The network's parameters — the weights and biases — are determined by a training algorithm. The output layer consists of $|C|$ neurons, each outputting the score of one of the labels. We denote the output logits by $\hat{y}(x) = z^{[L]}(x)$.

Our definitions focus on fully-connected layers, but our implementation also supports convolutional layers and it is easily extensible to other layer types such as max-pooling layers [3].

2.2 Confidence Margin

The *confidence margin* captures how certain a network is in its classification. Intuitively, a high confidence margin means the network assigns the correct class a score well above all other classes, making it harder for a perturbation to change the classification.

Definition 1 (Confidence margin). *Given a network N, an input x, and a class $c \in C$, the* class confidence *of N in c at x is:*

$$\mathcal{C}(N, x, c) \;:=\; \hat{y}_c(x) \;-\; \max_{k \neq c} \hat{y}_k(x).$$

The confidence $\mathcal{C}(N, x, c) > 0$ if and only if N classifies x as c.
Two-class setting. *When there are only two output classes c_{tag} and c_{target}, we write $\mathcal{C}(N, x)$ as shorthand for $\mathcal{C}(N, x, c_{\text{tag}}) = \hat{y}_{c_{\text{tag}}}(x) - \hat{y}_{c_{\text{target}}}(x)$.*

2.3 Robustness and Foolability

We next define robustness and foolability with respect to a perturbation set. These notions form the building blocks of both the standard-mode and transfer-mode problems.

Definition 2 ($(\varepsilon, c \to t)$-robustness). *Given a network N and a perturbation set $\mathcal{I}_\varepsilon(x)$ (defined per perturbation type in Section 3), an input x is* $(\varepsilon, c \to t)$-robust *for N if $\forall\, x' \in \mathcal{I}_\varepsilon(x) :\; N(x') \neq t$. That is, no perturbation in the allowed set causes N to misclassify x as the target class t.*

Definition 3 (N-foolability). *An input $x \in \mathcal{X}$ is* N-foolable *if there exists a perturbation $\varepsilon \in \mathcal{E}$ with $x + \varepsilon \in \mathcal{X}$ such that $\mathcal{C}(N, x + \varepsilon) \leq -\eta$, where $\eta > 0$ is a small attack margin (typically 10^{-3}). We write $\mathcal{F}_N$ for the set of all N-foolable inputs.*

Intuitively, an input is foolable if the adversary can find a perturbation that causes the network to misclassify it. The set $\mathcal{F}_N$ contains all inputs that are vulnerable to adversarial attack under the given perturbation model.

2.4 MIP Encoding of ReLU

Our approach encodes the network's computation as a mixed-integer program (MIP). The JuMP/Gurobi MIP [5,4] encodes each linear layer exactly and linearizes every ReLU via the standard big-M formulation with a binary variable $b \in \{0, 1\}$:

$$a \;=\; \max(0, z), \qquad 0 \leq a \leq M\, b, \qquad z \leq a, \qquad a \leq z + M(1 - b).$$

This encoding assigns each non-input ReLU neuron a binary variable, whose domain is $\{0, 1\}$. Generally, the time complexity of a MIP problem over boolean and real variables is exponential in the number of boolean variables. To make the analysis more efficient, several works propose ways to identify boolean variables that can be removed (e.g., ReLUs whose inputs are non-positive or non-negative), thereby reducing the complexity [8].

2.5 Formal Definitions of the Central Quantities

We now formalize the three central MIP quantities. Throughout, we use: $\mathcal{X} = [0,1]^n$ (input domain), $\mathcal{E} = [-p,p]^n$ (perturbation ball for radius $p > 0$), attack margin $\eta > 0$, and transition margin $\varepsilon_0 > 0$ (both typically 10^{-3}).

Definition 4 (Standard-mode value). *The* standard-mode value *of network N_i is:*

$$\delta_1(N_i) \;:=\; \sup\big\{\, \mathcal{C}(N_i,x) \;\big|\; x \in \mathcal{F}_i,\; \mathcal{C}(N_i,x) \geq 0 \,\big\}.$$

Intuitively, this is the maximum confidence N_i attains on any input that is correctly classified yet can still be misclassified by some perturbation $\varepsilon \in \mathcal{E}$. Any input with confidence above $\delta_1(N_i)$ is provably robust.

Definition 5 (Transfer MIP — original). *Given networks N_1 and N_2 with pre-computed $\delta_1(N_1)$, the* transfer value *is:*

$$\begin{aligned}\delta^*_{\text{diff}} \;:=\; \sup\Big\{\, \mathcal{C}(N_2,x) - \mathcal{C}(N_1,x) \;\Big|\; & \\ &\text{(C1) } \mathcal{C}(N_1,x) \geq \delta_1(N_1) + \varepsilon_0, \\ &\text{(C2) } \mathcal{C}(N_2,x) \geq \mathcal{C}(N_1,x), \\ &\text{(C3) } x \in \mathcal{F}_2 \,\Big\}. \end{aligned} \tag{1}$$

Constraint C1 ensures x lies in the region where N_1 is certifiably robust. C2 ensures N_2 is at least as confident as N_1. C3 ensures a perturbation exists that fools N_2. The objective maximizes the confidence gap, finding the worst-case scenario for the transfer guarantee.

Definition 6 (Modified transfer MIP).

$$\begin{aligned}\hat{\delta}_{\text{diff}} \;:=\; \sup\Big\{\, \mathcal{C}(N_2,x) - \mathcal{C}(N_1,x) \;\Big|\; & \\ &\text{(}\widehat{\text{C1}}\text{) } x \in \mathcal{F}_1, \\ &\text{(C3) } x \in \mathcal{F}_2 \,\Big\}. \end{aligned} \tag{2}$$

*The key change: C1 is replaced by $\widehat{\text{C1}}$, which requires x to be N_1-*foolable *rather than N_1-robust.*

Definition 7 (Joint-foolable set). *$\mathcal{F}_{1\wedge 2} := \mathcal{F}_1 \cap \mathcal{F}_2$: the set of inputs simultaneously foolable by both networks.*

Definition 8 (Modified standard-mode value).

$$\hat{\delta}_1(N_2) \;:=\; \sup\big\{\, \mathcal{C}(N_2,x) \;\big|\; x \in \mathcal{F}_{1\wedge 2},\; \mathcal{C}(N_2,x) \geq 0 \,\big\}.$$

This is the maximum confidence N_2 achieves at inputs simultaneously foolable by both *networks. By definition, $\hat{\delta}_1(N_2) \leq \delta_1(N_2)$, since restricting to the joint-foolable set can only decrease the supremum.*

3 Perturbation Models

In this section, we define the perturbation models supported by our framework. A perturbation is a function $f_P(x, \mathcal{E})$, defining how an input x is perturbed given a perturbation amount specified by a vector $\mathcal{E}$. All perturbations are parameterized by a scalar $\varepsilon > 0$ bounding their magnitude. For each perturbation type, we also define the *perturbation-difference interval* $[I^{\text{dn}}, I^{\text{up}}] \in \mathbb{R}^n \times \mathbb{R}^n$, which captures the element-wise range of $x' - x$ at the input layer. These intervals are propagated layer-by-layer to tighten the MIP (Section 8.2).

3.1 ℓ_∞ Perturbation

The ℓ_∞ perturbation has a perturbation limit $\varepsilon \in [0, 1]$ and is defined by a series of perturbations $\varepsilon_i \in [-\varepsilon, \varepsilon]$, for $i \in [n]$. The perturbation set is:

$$\mathcal{I}_\varepsilon(x) = \left\{x' : \|x' - x\|_\infty \leq \varepsilon,\ x' \in [0,1]^n\right\}.$$

In the MIP encoding, this translates to: $x_i - \varepsilon \leq x'_i \leq x_i + \varepsilon$ for every pixel i, with clipping to $[0, 1]$.

The perturbation-difference interval is: $I_i^{\text{dn}} = -\varepsilon$, $I_i^{\text{up}} = +\varepsilon$ for all i.

3.2 Brightness Perturbation

Brightness is defined by a brightness level $e \in [0, \varepsilon]$ such that $x'_i = x_i + e$ for all pixels i. The perturbation set is:

$$\mathcal{I}_\varepsilon(x) = \left\{x' : x' = x + e\,\mathbf{1},\ e \in [0, \varepsilon]\right\}.$$

In the MIP encoding, we add a scalar variable $e \in [0, \varepsilon]$ and set $x'_i = x_i + e$.

Since $e \geq 0$, the brightness shift is non-negative. The perturbation-difference interval is therefore: $I_i^{\text{dn}} = 0$, $I_i^{\text{up}} = +\varepsilon$ for all i.

3.3 Contrast Perturbation

Contrast is defined by a scaling factor $e \in [1, 1 + \varepsilon]$ such that $x'_i = e \cdot x_i$. The perturbation set is:

$$\mathcal{I}_\varepsilon(x) = \left\{x' : x' = e \cdot x,\ e \in [1, 1 + \varepsilon]\right\}.$$

In the MIP encoding, we add a scalar variable $e \in [1, 1+\varepsilon]$ and set $x'_i = e \cdot x_i$. Note that this introduces a bilinear constraint, which requires the Gurobi non-convex mode.

Since $e \geq 1$ and $x_i \geq 0$, we have $x'_i - x_i = (e-1)x_i \in [0, \varepsilon]$. The perturbation-difference interval is an over-approximation: $I_i^{\text{dn}} = 0$, $I_i^{\text{up}} = +\varepsilon$ for all i.

3.4 Translation Perturbation

Translation is defined by shift coordinates (d_h, d_w) with $|d_h|, |d_w| \leq k$ pixels, and vacated positions are zero-padded. The shift direction is selected by binary variables $b_h, b_w \in \{0, 1\}$ and offset variables $d_h, d_w \in [0, k]$.

Pixel values lie in $[0, 1]$. After translation, the worst-case difference at any pixel is $x'_i - x_i \in [-x_i, 1 - x_i] \subseteq [-1, 1]$. We use the conservative perturbation-difference interval: $I_i^{\text{dn}} = -1$, $I_i^{\text{up}} = +1$ for all i.

3.5 Patch Perturbation

A rectangular patch $P \subseteq [n]$ of pixels is perturbed by $\pm\varepsilon$, while pixels outside P are unchanged:

$$x'_i = x_i + e_i\,\varepsilon, \quad e_i \in [-1, +1], \quad i \in P; \qquad x'_i = x_i, \quad i \notin P.$$

The perturbation-difference interval reflects this structure:

$$I_i^{\text{dn}} = \begin{cases} -\varepsilon & i \in P \\ 0 & i \notin P, \end{cases} \qquad I_i^{\text{up}} = \begin{cases} +\varepsilon & i \in P \\ 0 & i \notin P. \end{cases}$$

3.6 Occlusion Perturbation

An occlusion perturbation sets a rectangular patch P to zero, while pixels outside P are unchanged:

$$x'_i = 0, \quad i \in P; \qquad x'_i = x_i, \quad i \notin P.$$

Because $x_i \in [0, 1]$ and $x'_i = 0$ inside P, the difference $x'_i - x_i = -x_i \in [-1, 0]$. Outside P, the difference is exactly zero. The perturbation-difference interval is therefore:

$$I_i^{\text{dn}} = \begin{cases} -1 & i \in P \\ 0 & i \notin P, \end{cases} \qquad I_i^{\text{up}} = 0, \quad \forall\, i.$$

3.7 Summary

Table 1 summarizes the perturbation sets and their corresponding input-layer difference intervals.

4 Standard Mode: Computing the Global Robustness Bound

In this section, we present the standard-mode formulation, which computes the global robustness bound for a single network. This bound serves as the foundation for the transfer-mode analysis in Section 5.

Table 1. Summary of perturbation sets and input-layer difference intervals.

Perturbation	**Perturbation set** $\mathcal{I}_\varepsilon(x)$	I_i^{dn}	I_i^{up}
ℓ_∞	$\|x' - x\|_\infty \le \varepsilon$	$-\varepsilon$	$+\varepsilon$
Brightness	$x' = x + e$, $e \in [0, \varepsilon]$	0	$+\varepsilon$
Contrast	$x' = e \cdot x$, $e \in [1, 1+\varepsilon]$	0	$+\varepsilon$
Translation	pixel shift $\le k$ px, zero-pad	-1	$+1$
Patch	$\pm\varepsilon$ inside patch P	$-\varepsilon \cdot \mathbf{1}_P$	$+\varepsilon \cdot \mathbf{1}_P$
Occlusion	zero inside patch P	$-\mathbf{1}_P$	0

4.1 Problem Definition

We fix a source class c and a target class t. The standard-mode MIP maximizes the confidence margin of the *clean* input x while requiring that a *perturbed* input x' is misclassified as t:

$$\delta_1 \;=\; \max_{x,\,x'} \mathcal{C}(N_1, x, c) \quad \text{subject to} \quad x' \in \mathcal{I}_\varepsilon(x), \quad \hat{y}_t(x') \ge \hat{y}_k(x')\ \forall k \ne t. \tag{3}$$

Intuitively, the solution δ_1 is the highest confidence at which the network can still be fooled. Any input classified with confidence above δ_1 is provably robust: no perturbation in $\mathcal{I}_\varepsilon(x)$ can cause misclassification as t. This is equivalent to Definition 4: $\delta_1 = \delta_1(N_1)$ in the formal notation.

4.2 MIP Encoding of the Confidence Margin

The confidence margin involves $\max_{k \ne c} \hat{y}_k$, which is non-linear. We linearize it by introducing one binary variable $b_k \in \{0, 1\}$ per class $k \ne c$, indicating whether class k achieves the maximum:

$$\begin{aligned} \delta &= \hat{y}_c - m, \\ m &\ge \hat{y}_k \quad \forall\, k \ne c, \\ m &\le \hat{y}_k + M(1 - b_k) \quad \forall\, k \ne c, \\ \sum_{k \ne c} b_k &= 1, \end{aligned}$$

where M is a sufficiently large constant. The solver then maximizes δ. This encoding is exact: the binary variables ensure that m equals the maximum logit among all non-c classes, and the MIP objective finds the maximum confidence at which an adversarial perturbation exists.

5 Transfer Mode: Verifying Robustness Across Networks

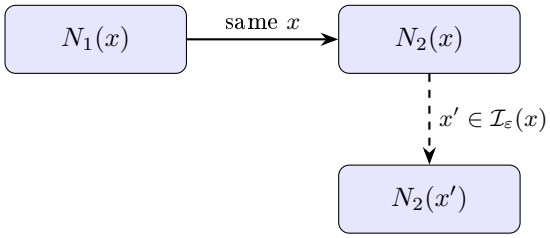
In this section, we present our main contribution: the transfer-mode formulation. We begin by motivating the problem, then describe our three-network MIP encoding, state the transfer MIP, and prove its correctness.

5.1 Motivation

Training a more accurate network N_2 from N_1 (e.g., via distillation or fine-tuning) often increases prediction confidence on correctly classified inputs. A natural question is: *by how much can N_2's confidence exceed N_1's in the worst case, in the regime where N_1 is already certifiably robust but N_2 can still be fooled?* The answer to this question determines whether N_1's robustness guarantee transfers to N_2.

5.2 Three-Network Encoding

To answer this question, the transfer mode encodes **three** forward passes simultaneously in a single MIP: $N_1(x)$, $N_2(x)$, and $N_2(x')$.



Each forward pass uses a unique variable-name prefix to distinguish the three copies. The clean input x is shared between $N_1(x)$ and $N_2(x)$, and the perturbed input x' is linked to x via the perturbation constraint. Note that $N_1(x')$ is *not* encoded: the transfer MIP only requires N_1's confidence on the clean input to establish the robustness baseline, and N_2's computation on both the clean and perturbed inputs to verify whether the attack succeeds. This saves all of the ReLU binary variables that would be needed for a fourth network pass, significantly reducing the problem size.

5.3 Transfer MIP Formulation

Let $c_1 = \mathcal{C}(N_1, x, c)$, $c_2 = \mathcal{C}(N_2, x, c)$, and $c_2' = \mathcal{C}(N_2, x', c)$. The transfer MIP is:

$$\delta_{\text{diff}} \;=\; \max_{x,\,x',\,e} \;(c_2 - c_1) \quad \text{s.t.} \quad \begin{cases} x' \in \mathcal{I}_\varepsilon(x) & \text{(perturbation constraint)} \\ c_1 \;\geq\; \delta_1 + \epsilon_0 & \text{(T1: } N_1 \text{ is certifiably robust)} \\ c_2 - c_1 \;\geq\; 0 & \text{(T2: } N_2 \text{ at least as confident as } N_1\text{)} \\ \text{attack constraint on } N_2(x') & \text{(T3: } N_2 \text{ can be fooled)} \end{cases} \tag{4}$$

where $\epsilon_0 = 10^{-3}$ is a strict-inequality guard. In the formal notation of Section 2.5, this is exactly the transfer MIP of Definition 5, with $\delta_{\text{diff}} = \delta^*_{\text{diff}}$.

The constraint T1 restricts the search to inputs where N_1 is certifiably robust (its confidence exceeds the standard-mode bound δ_1). Constraint T2 focuses on the regime where N_2 is more confident than N_1. Constraint T3 requires that a perturbation exists that fools N_2, making the transfer problem non-trivial. The encoding of T3 depends on the chosen attack mode (Section 6).

5.4 Correctness of the Transfer Guarantee

We now prove that the transfer MIP provides a sound robustness certificate. The key insight is that any input violating the transfer guarantee would be a feasible solution to the MIP with objective value exceeding δ_{diff}, contradicting its optimality.

Theorem 1 (Transfer Robustness). *Let δ_1 be the standard-mode value for N_1 on perturbation $(\varepsilon, c \to t)$. Let δ_{diff} be the optimal value of the transfer MIP (4). Then for any input x:*

$$\mathcal{C}(N_1, x, c) \geq \delta_1 \text{ and } \mathcal{C}(N_2, x, c) - \mathcal{C}(N_1, x, c) > \delta_{\text{diff}} \implies x \text{ is } (\varepsilon, c \to t)\text{-robust for } N_2.$$

Proof. Suppose for contradiction that x is *not* $(\varepsilon, c \to t)$-robust for N_2. Then there exists $x' \in \mathcal{I}_\varepsilon(x)$ such that $N_2(x') = t$, meaning the attack constraint T3 holds.

Since $\mathcal{C}(N_1, x, c) \geq \delta_1$, constraint T1 is satisfied (with the ϵ_0 guard). Since $\mathcal{C}(N_2, x, c) - \mathcal{C}(N_1, x, c) > \delta_{\text{diff}} \geq 0$, constraint T2 is also satisfied.

Therefore (x, x') is a feasible solution to the transfer MIP (4) with objective value $c_2 - c_1 > \delta_{\text{diff}}$, contradicting the fact that δ_{diff} is the maximum feasible objective value.

Corollary 1 (Single-threshold inference). *Define $\Delta := \delta_1 + \delta_{\text{diff}}$. If $\mathcal{C}(N_1, x, c) \geq \delta_1$ and $\mathcal{C}(N_2, x, c) > \Delta$, then x is $(\varepsilon, c \to t)$-robust for N_2.*

This corollary shows that at inference time, the transfer guarantee reduces to checking two confidence thresholds, requiring only two forward passes and no additional optimization.

Remark 1. Checking $\mathcal{C}(N_2, x, c) > \Delta$ alone (without verifying N_1's threshold) is *unsound*: constraint T1 would not be guaranteed, making the transfer MIP potentially infeasible and δ_{diff} meaningless. Both conditions must be verified.

5.5 Relationship to the Formal Quantities

The transfer MIP (4) computes $\delta_{\text{diff}} = \delta^*_{\text{diff}}$, which satisfies $\delta_1(N_2) \geq \delta_1(N_1) + \delta^*_{\text{diff}}$ (Theorem 2 in Section 7). This is a *lower bound*: knowing δ_{diff} certifies that N_2's standard-mode value is at least $\delta_1 + \delta_{\text{diff}}$, but N_2's actual standard-mode value may be strictly larger. The precise gap and a counterexample are analyzed in Section 7.

6 Attack Constraint Variants

In this section, we describe the two variants for encoding constraint T3 ("N_2 is fooled by x'") in the transfer MIP. The choice of variant determines whether the adversary targets a specific class or any misclassification.

6.1 Targeted Attack

In the targeted variant, the adversary forces N_2 to predict a specific target class t. Formally, the constraint requires:

$$\hat{y}_t\big(N_2(x')\big) \;\geq\; \hat{y}_k\big(N_2(x')\big) \quad \forall\, k \neq t, \tag{5}$$

equivalently $\mathcal{C}(N_2, x', t) \geq 0$. This is the more restrictive variant: the adversary must not only cause a misclassification but must steer the network toward a particular wrong class.

6.2 Untargeted Attack

In the untargeted variant, the adversary need only reduce N_2's confidence in the true class c to negative, causing any misclassification:

$$\mathcal{C}(N_2, x', c) \;\leq\; 0. \tag{6}$$

6.3 Comparison

Table 2 compares the two variants. The targeted variant has a smaller feasible region (the adversary must achieve a specific misclassification), leading to a larger or equal δ_{diff} and thus a stronger transfer guarantee. The untargeted variant has a larger feasible region, yielding a smaller or equal δ_{diff} and a weaker but more general guarantee.

Table 2. Comparison of targeted and untargeted attack constraints.

Property	**Targeted**	**Untargeted**
Attack type	$c \to t$ (specific class)	$c \to$ any
Constraint on $N_2(x')$	$\hat{y}_t \geq \hat{y}_k,\ \forall k \neq t$	$\hat{y}_c < \hat{y}_k$ for some k
Feasible region	Smaller (harder to satisfy)	Larger (easier)
Resulting δ_{diff}	Larger or equal	Smaller or equal
Inference strength	Stronger guarantee	Weaker guarantee
Typical use	Specific adversary model	General robustness

7 Formal Analysis

In this section, we provide a rigorous analysis of the relationship between the standard-mode values $\delta_1(N_1)$, $\delta_1(N_2)$, and the transfer quantity δ^*_{diff}. We prove that the transfer MIP provides a sound lower bound, exhibit a counterexample showing the bound is strict, analyze the root cause of the gap, and propose a modified formulation with a matching upper bound. Throughout this section, we use the two-class confidence $\mathcal{C}(N, x) = \hat{y}_{c_{\text{tag}}}(x) - \hat{y}_{c_{\text{target}}}(x)$ for compactness.

7.1 The Transfer MIP Provides a Lower Bound

We first show that the transfer value δ^*_{diff} provides a sound lower bound on the improvement in robustness from N_1 to N_2. Intuitively, any feasible point of the transfer MIP is also a feasible point of the standard-mode problem for N_2, so the transfer MIP explores a subset of the relevant search space.

Theorem 2 (Lower bound on $\delta_1(N_2)$). *If the transfer MIP* (1) *is feasible, then*

$$\boxed{\delta_1(N_2) \;\geq\; \delta_1(N_1) + \delta^*_{\text{diff}}.}$$

Proof. Fix $\epsilon > 0$ and let x^* be a nearly-optimal feasible point of (1), with ε^* the corresponding attack perturbation (from C3). By feasibility:

$$\mathcal{C}(N_1, x^*) \geq \delta_1(N_1) + \varepsilon_0, \tag{C1}$$
$$\mathcal{C}(N_2, x^*) \geq \mathcal{C}(N_1, x^*), \tag{C2}$$
$$x^* \in \mathcal{F}_2 \text{ via } \varepsilon^*, \tag{C3}$$
$$\mathcal{C}(N_2, x^*) - \mathcal{C}(N_1, x^*) \geq \delta^*_{\text{diff}} - \epsilon. \tag{obj}$$

Step 1. We lower-bound $\mathcal{C}(N_2, x^*)$:

$$\begin{aligned}\mathcal{C}(N_2, x^*) &= \mathcal{C}(N_1, x^*) + \big(\mathcal{C}(N_2, x^*) - \mathcal{C}(N_1, x^*)\big) \\ &\geq (\delta_1(N_1) + \varepsilon_0) + (\delta^*_{\text{diff}} - \epsilon) \geq \delta_1(N_1) + \delta^*_{\text{diff}} - \epsilon.\end{aligned}$$

Step 2. We show x^* is feasible for the standard-mode MIP on N_2. By C2 and C1, $\mathcal{C}(N_2, x^*) \geq \mathcal{C}(N_1, x^*) \geq \delta_1(N_1) + \varepsilon_0 > 0$, so N_2 correctly classifies x^*. Combined with C3 ($x^* \in \mathcal{F}_2$), the pair (x^*, ε^*) is feasible for the standard-mode MIP on N_2. Hence:

$$\delta_1(N_2) \;\geq\; \mathcal{C}(N_2, x^*) \;\geq\; \delta_1(N_1) + \delta^*_{\text{diff}} - \epsilon.$$

Since $\epsilon > 0$ is arbitrary, we conclude $\delta_1(N_2) \geq \delta_1(N_1) + \delta^*_{\text{diff}}$.

Remark 2 (The gap is always positive). Let x^* achieve the supremum in (1). The proof shows:

$$\delta_1(N_2) - (\delta_1(N_1) + \delta^*_{\text{diff}}) \;\geq\; \mathcal{C}(N_1, x^*) - \delta_1(N_1) \;\geq\; \varepsilon_0 \;>\; 0.$$

The gap is always positive because constraint C1 forces $\mathcal{C}(N_1, x^*) > \delta_1(N_1)$. This surplus $\mathcal{C}(N_1, x^*) - \delta_1(N_1)$ is the precise "slack" that makes the theorem strict.

7.2 The Reverse Inequality Fails: A Counterexample

We next show that the reverse inequality, $\delta_1(N_1) + \delta^*_{\text{diff}} \geq \delta_1(N_2)$, is false in general. We exhibit two explicit linear networks that produce a strict gap.

Network Definitions We consider two linear networks over $\mathcal{X} = [0,1]^2$, with perturbation radius $p = 0.1$, and margins $\eta = \varepsilon_0 = 10^{-3}$.

Network	Logits	Confidence margin $\mathcal{C}(N,\cdot)$
N_1	$[x_1,\ x_2]$	$x_1 - x_2$
N_2	$[2x_1,\ 5x_2]$	$2x_1 - 5x_2$

Both are two-layer networks: an identity first layer, an inactive ReLU (since $x_1, x_2 \geq 0$), and a diagonal second layer ($\mathrm{diag}(1,1)$ for N_1, $\mathrm{diag}(2,5)$ for N_2). Despite their simplicity, these networks suffice to demonstrate that the gap can be substantial.

Foolability Conditions We derive the foolability conditions for each network.

N_1 is fooled at $x + \varepsilon$ if and only if $(x_2 + \varepsilon_2) - (x_1 + \varepsilon_1) \geq \eta$, i.e., $\varepsilon_2 - \varepsilon_1 \geq (x_1 - x_2) + \eta$. The maximum of $\varepsilon_2 - \varepsilon_1$ over $\mathcal{E}$ is $2p = 0.2$, so:

$$x \in \mathcal{F}_1 \iff x_1 - x_2 \ \leq\ 2p - \eta \ =\ 0.199.$$

N_2 is fooled at $x+\varepsilon$ if and only if $5(x_2+\varepsilon_2)-2(x_1+\varepsilon_1) \geq \eta$, i.e., $5\varepsilon_2 - 2\varepsilon_1 \geq (2x_1 - 5x_2) + \eta$. The maximum of $5\varepsilon_2 - 2\varepsilon_1$ over $\mathcal{E}$ is $5p + 2p = 0.7$, so:

$$x \in \mathcal{F}_2 \iff 2x_1 - 5x_2 \ \leq\ 5p + 2p - \eta \ =\ 0.699.$$

Computing the Quantities We now compute $\delta_1(N_1)$, $\delta_1(N_2)$, and δ^*_{diff} for our counterexample networks.

Standard-mode value $\delta_1(N_1)$. We maximize $x_1 - x_2$ subject to $x_1 - x_2 \leq 0.199$ and $x \in [0,1]^2$: $\delta_1(N_1) = 0.199$, achieved at $x^{(1)} = (1,\ 0.801)$.

Standard-mode value $\delta_1(N_2)$. We maximize $2x_1 - 5x_2$ subject to $2x_1 - 5x_2 \leq 0.699$ and $x \in [0,1]^2$: $\delta_1(N_2) = 0.699$, achieved at $x^{(2)} = (1,\ 0.2602)$ with attack $\varepsilon^* = (-0.1, +0.1)$.

Transfer value δ^*_{diff}. The objective is $\mathcal{C}(N_2, x) - \mathcal{C}(N_1, x) = (2x_1 - 5x_2) - (x_1 - x_2) = x_1 - 4x_2$. The constraints are:

$$\begin{aligned} &\text{C1}: \quad x_1 - x_2 \geq 0.200, \\ &\text{C2}: \quad x_1 - 4x_2 \geq 0, \\ &\text{C3}: \quad 2x_1 - 5x_2 \leq 0.699. \end{aligned}$$

Setting $x_2 = 0$ (which minimizes the $-4x_2$ penalty), C1 gives $x_1 \geq 0.200$ and C3 gives $x_1 \leq 0.3495$. Maximizing x_1 yields:

$$x^* = (0.3495,\ 0), \qquad \delta^*_{\text{diff}} = x^*_1 - 4 \cdot 0 = \mathbf{0.3495}.$$

Note that $\mathcal{C}(N_1, x^*) = 0.3495$, which is substantially larger than $\delta_1(N_1) = 0.199$. This is the source of the gap.

The Counterexample

Quantity	Value
$\delta_1(N_1)$	0.199
δ^*_{diff}	0.3495
$\delta_1(N_1) + \delta^*_{\text{diff}}$	**0.5485**
$\delta_1(N_2)$	**0.699**
Gap: $\delta_1(N_2) - (\delta_1(N_1) + \delta^*_{\text{diff}})$	**0.1505**

$$\delta_1(N_1) + \delta^*_{\text{diff}} \;=\; 0.5485 \;<\; 0.699 \;=\; \delta_1(N_2).$$

The gap equals exactly $\mathcal{C}(N_1, x^*) - \delta_1(N_1) = 0.3495 - 0.199 = 0.1505$, consistent with Remark 2.

7.3 Root-Cause Analysis

Theorem 2 and the counterexample together establish that $\delta_1(N_1) + \delta^*_{\text{diff}}$ is a *strict lower bound* on $\delta_1(N_2)$, with gap:

$$\text{gap} \;=\; \underbrace{\mathcal{C}(N_1, x^*)}_{\geq\, \delta_1(N_1)+\varepsilon_0 \text{ (C1)}} - \delta_1(N_1) \;\geq\; \varepsilon_0 \;>\; 0.$$

The structural reason is the following. The transfer MIP searches for an x^* that simultaneously satisfies C1 (high N_1-confidence) and C3 (N_2-foolable). Because N_1 and N_2 have *different* decision boundaries, the N_2-foolable region can contain points with N_1-confidence far above $\delta_1(N_1)$. At such an x^*, the quantity $\mathcal{C}(N_2, x^*) = \mathcal{C}(N_1, x^*) + \delta^*_{\text{diff}}$ is large, but the formula $\delta_1(N_1) + \delta^*_{\text{diff}}$ loses the surplus $\mathcal{C}(N_1, x^*) - \delta_1(N_1)$.

Equivalently, the transfer MIP's feasible set $\{x : \text{C1+C3}\}$ is a *proper subset* of the standard-mode feasible set for N_2 ($\{x \in \mathcal{F}_2\}$):

$$\delta_1(N_2) \;=\; \sup_{x \in \mathcal{F}_2} \mathcal{C}(N_2, x) \;\geq\; \sup_{x:\, \text{C1+C3}} \mathcal{C}(N_2, x) \;\geq\; \sup_{x:\, \text{C1+C2+C3}} \left[\mathcal{C}(N_1, x) + \delta^*_{\text{diff}}\right] \;\geq\; \delta_1(N_1) + \delta^*_{\text{diff}}.$$

The first inequality can be strict because $\mathcal{F}_2$ contains points with high $\mathcal{C}(N_2, \cdot)$ that *violate* C1 (i.e., they are not N_1-robust).

7.4 A Modified Formulation with a Matching Upper Bound

Motivation Remark 2 reveals that the gap is driven by C1 forcing $\mathcal{C}(N_1, x^*) > \delta_1(N_1)$. To close the gap, we replace C1 with its logical opposite: instead of requiring N_1 to be *robust* at x, we require N_1 to be *foolable* at x. This simultaneously restricts the standard-mode problem for N_2 to the same joint-foolability region.

The modified transfer MIP is Definition 6, with modified standard-mode value $\hat{\delta}_1(N_2)$ (Definition 8). Note that $\hat{\delta}_1(N_2) \leq \delta_1(N_2)$ by definition: restricting to $\mathcal{F}_{1\wedge 2} \subseteq \mathcal{F}_2$ can only decrease the supremum. The gap $\delta_1(N_2) - \hat{\delta}_1(N_2)$ measures the advantage N_2 has at points where it is foolable but N_1 is not — the "exclusive vulnerability" of N_2.

MIP Encoding The constraint $\widehat{\text{C1}}$ introduces a second perturbation variable $\varepsilon^{(1)} \in \mathcal{E}$ with the foolability constraint $\mathcal{C}(N_1, x + \varepsilon^{(1)}) \leq -\eta$, encoded exactly as in the standard-mode MIP for N_1. Constraint C3 is encoded as before. The resulting program has $O(|N_1|+|N_2|)$ binary variables and is solvable by standard MIP solvers.

Main Result

Theorem 3 (Upper bound on the joint standard-mode value).

$$\boxed{\delta_1(N_1) + \hat{\delta}_{\text{diff}} \;\geq\; \hat{\delta}_1(N_2).}$$

Proof. Fix $\epsilon > 0$ and let x^{**} be a nearly-optimal feasible point of $\hat{\delta}_1(N_2)$ (Definition 8), i.e., $x^{**} \in \mathcal{F}_{1\wedge 2}$, $\mathcal{C}(N_2, x^{**}) \geq 0$, $\mathcal{C}(N_2, x^{**}) \geq \hat{\delta}_1(N_2) - \epsilon$.

Step 1. Since $x^{**} \in \mathcal{F}_1$, by Definition 4, $\mathcal{C}(N_1, x^{**}) \leq \delta_1(N_1)$.

Step 2. Since $x^{**} \in \mathcal{F}_1$ satisfies $\widehat{\text{C1}}$ and $x^{**} \in \mathcal{F}_2$ satisfies C3, x^{**} is feasible for (2). Hence: $\hat{\delta}_{\text{diff}} \;\geq\; \mathcal{C}(N_2, x^{**}) - \mathcal{C}(N_1, x^{**})$.

Step 3. Combining:

$$\begin{aligned}
\delta_1(N_1) + \hat{\delta}_{\text{diff}} &\geq \delta_1(N_1) + \mathcal{C}(N_2, x^{**}) - \mathcal{C}(N_1, x^{**}) \\
&\geq \mathcal{C}(N_1, x^{**}) + \mathcal{C}(N_2, x^{**}) - \mathcal{C}(N_1, x^{**}) \quad \text{(Step 1: } \delta_1(N_1) \geq \mathcal{C}(N_1, x^{**})\text{)} \\
&= \mathcal{C}(N_2, x^{**}) \;\geq\; \hat{\delta}_1(N_2) - \epsilon.
\end{aligned}$$

Since $\epsilon > 0$ is arbitrary, $\delta_1(N_1) + \hat{\delta}_{\text{diff}} \geq \hat{\delta}_1(N_2)$.

Corollary 2 (Sandwich bounds). *If both transfer MIPs are feasible:*

$$\delta_1(N_1) + \delta^*_{\text{diff}} \;\leq\; \delta_1(N_2), \qquad\qquad \delta_1(N_1) + \hat{\delta}_{\text{diff}} \;\geq\; \hat{\delta}_1(N_2).$$

Remark 3 (Complementary regions). The two transfer quantities explore complementary regions: the original δ^*_{diff} searches over N_1-*robust* $\cap$ N_2-*foolable* inputs, while $\hat{\delta}_{\text{diff}}$ searches over N_1-*foolable* $\cap$ N_2-*foolable* inputs. These regions are disjoint. In particular, $\hat{\delta}_{\text{diff}}$ can be negative (if N_1 always has higher confidence than N_2 in $\mathcal{F}_{1\wedge 2}$), while $\delta^*_{\text{diff}} \geq 0$ by constraint C2.

7.5 Numerical Verification

We verify Theorem 3 on the counterexample networks from Section 7.2.

Modified transfer value $\hat{\delta}_{\text{diff}}$. We maximize $x_1 - 4x_2$ subject to $x_1 - x_2 \leq 0.199$ (N_1-foolable) and $2x_1 - 5x_2 \leq 0.699$ (N_2-foolable), with $x \in [0,1]^2$. Setting $x_2 = 0$: $x_1 \leq \min(0.199,\, 0.3495) = 0.199$. This yields $\hat{\delta}_{\text{diff}} = \mathbf{0.199}$.

Modified standard-mode value $\hat{\delta}_1(N_2)$. We maximize $2x_1 - 5x_2$ subject to both foolability constraints. At $x_2 = 0$: $x_1 \leq 0.199$, so $\hat{\delta}_1(N_2) = 2 \times 0.199 = \mathbf{0.398}$.

Quantity	Value
$\delta_1(N_1)$	0.199
$\hat{\delta}_{\text{diff}}$	0.199
$\delta_1(N_1) + \hat{\delta}_{\text{diff}}$	**0.398**
$\hat{\delta}_1(N_2)$	**0.398**
$\delta_1(N_1) + \hat{\delta}_{\text{diff}} \geq \hat{\delta}_1(N_2)$?	**YES** (equality holds)
$\delta_1(N_2)$ (unrestricted)	0.699
Joint gap: $\delta_1(N_2) - \hat{\delta}_1(N_2)$	0.301

Equality holds in Theorem 3 because the binding constraint is simultaneously $\widehat{\text{C1}}$ and C3, making x^{**} the argmax for both $\hat{\delta}_1(N_2)$ and $\hat{\delta}_{\text{diff}}$. The joint gap of 0.301 reflects N_2's foolable region that is N_1-robust — precisely the regime probed by the original transfer MIP.

7.6 Discussion: Two Complementary Formulations

The original and modified transfer MIPs provide complementary views of the robustness relationship between two networks. Table 3 summarizes their characteristics.

Table 3. Comparison of the two transfer MIP formulations.

Formulation	**Feasible region**	**Bound direction**
Original δ^*_{diff}	N_1-robust $\cap$ N_2-foolable	$\delta_1(N_2) \geq \delta_1(N_1) + \delta^*_{\text{diff}}$ (lower bound)
Modified $\hat{\delta}_{\text{diff}}$	N_1-foolable $\cap$ N_2-foolable	$\delta_1(N_1) + \hat{\delta}_{\text{diff}} \geq \hat{\delta}_1(N_2)$ (upper bound)

The original formulation asks: "By how much can N_2's confidence exceed N_1's, at a point where N_1 is provably robust?" Theorem 2 shows this provides a lower bound on $\delta_1(N_2) - \delta_1(N_1)$.

The modified formulation asks: "At points hardest for *both* networks simultaneously, how much more confident is N_2 than N_1?" Theorem 3 shows this provides an upper bound on $\hat{\delta}_1(N_2) - \delta_1(N_1)$.

Together, the two quantities bracket the robustness difference from both sides over their respective domains.

8 Tightening Techniques

In this section, we present four complementary techniques for tightening the MIP relaxation. As described in Section 4, the time complexity of the MIP is governed by the number of boolean variables. To enable the MIP solver to reach a solution efficiently, we introduce techniques that reduce the search space by providing tighter bounds and warm-start solutions. The four techniques can be combined freely, progressively trading preprocessing time for faster solver convergence.

Table 4. Overview of the four tightening techniques.

Technique	**Method**	**Preprocess**	**Benefit**
Hyper-adversarial attack	PGD warm-start	Python PGD	Incumbent cutoff
Perturbation-diff. intervals	Interval propagation	Interval arith.	Activation bounds
Inter-network intervals	Weight-difference bounds	Interval arith.	Cross-network bounds
Dependency analysis	Monotonicity signatures	Symbolic prop.	Ordering constraints

8.1 Hyper-Adversarial Attack

Our first technique expedites the tightening of the lower bound by efficiently computing a suboptimal feasible solution and providing it to the MIP solver. Before launching the MIP solver, we run a projected gradient descent (PGD) attack [9] to find a feasible solution $(x^\star, x'^\star)$ with objective value $v^\star$.

This solution is used in two ways. First, as an *incumbent cutoff*: the solver can prune any branch whose objective value cannot exceed $v^\star$. Formally, we set the Gurobi incumbent to $v^\star$, enabling the solver to discard large portions of the search tree.

Second, as a *binary warm-start*: for each ReLU binary variable b_j, we compute the activation sign at the PGD solution and provide it as a hint:

$$b_j^{\text{hint}} \;=\; \mathbf{1}\Big[z_j^{[l]}(x^\star) \geq 0\Big].$$

These hints guide the solver toward the optimal branching decisions, significantly reducing the number of explored nodes.

8.2 Perturbation-Difference Interval Propagation

Our second technique bounds the activation differences between the clean and perturbed forward passes. For each layer l, we compute an interval $[I^{\text{dn}[l]}, I^{\text{up}[l]}]$

bounding the element-wise difference $a^{\langle pert \rangle [l]} - a^{\langle org \rangle [l]}$. The same propagation applies to both the N_1 and N_2 network pairs.

Initialization (input layer). From Table 1: $I^{\mathrm{dn}[0]} = I^{\mathrm{dn}}$, $I^{\mathrm{up}[0]} = I^{\mathrm{up}}$.

Linear layer propagation. For weight matrix $W^{[l]}$ (the bias cancels in the difference):

$$I^{\mathrm{up}[l]} = (W^{[l]})^{+} I^{\mathrm{up}[l-1]} + (W^{[l]})^{-} I^{\mathrm{dn}[l-1]}, \tag{7}$$

$$I^{\mathrm{dn}[l]} = (W^{[l]})^{+} I^{\mathrm{dn}[l-1]} + (W^{[l]})^{-} I^{\mathrm{up}[l-1]}, \tag{8}$$

where $W^{+} = \max(W, 0)$ and $W^{-} = \min(W, 0)$.

ReLU layer propagation. We use a linear over-approximation:

$$I^{\mathrm{up}[l]} \leftarrow \max\big(0,\, I^{\mathrm{up}[l-1]}\big), \qquad I^{\mathrm{dn}[l]} \leftarrow \min\big(0,\, I^{\mathrm{dn}[l-1]}\big).$$

Added MIP constraints. At every layer l and neuron j:

$$a^{\langle org \rangle [l]}_{j} + I^{\mathrm{dn}[l]}_{j} \;\le\; a^{\langle pert \rangle [l]}_{j} \;\le\; a^{\langle org \rangle [l]}_{j} + I^{\mathrm{up}[l]}_{j}. \tag{9}$$

These constraints couple the clean and perturbed forward passes, tightening the LP relaxation. We next prove their soundness.

Soundness, completeness, and tightness. We distinguish three properties of added constraints in a MIP verifier:

- **Soundness** (no false negatives): a constraint is sound if it is satisfied by every feasible point of the original MIP, so it never removes a real solution.
- **Completeness** (no false positives): a constraint is complete if it does not introduce spurious feasible points, so the LP relaxation does not admit solutions that could not arise from any true (x, x') pair.
- **Tightness** (quality of approximation): a sound constraint is tight at a point if the bound it provides equals the true value there.

The interval constraints (9) are **sound and complete but not always tight**. They are sound because the propagated intervals are genuine over-approximations (Theorem 4). They are complete because, being over-approximations, they can only widen the set of allowed activations. They are not always tight because the ReLU propagation step uses a linear over-approximation that is loose when one of the two compared activations is negative and the other is positive.

Theorem 4 (Soundness of perturbation-difference interval propagation). *Let N be a ReLU network with weights $W^{[l]}$, biases $b^{[l]}$. Fix any x and x' such that $x'_i - x_i \in [I_i^{\mathrm{dn}[0]},\, I_i^{\mathrm{up}[0]}]$ elementwise. Then for every layer l and every neuron j:*

$$a_j^{[l]}(x') - a_j^{[l]}(x) \;\in\; \big[I_j^{\mathrm{dn}[l]},\, I_j^{\mathrm{up}[l]}\big],$$

where $[I^{\mathrm{dn}[l]}, I^{\mathrm{up}[l]}]$ are computed by the recursion in (7)–(8) and the ReLU propagation step.

Proof. By induction on l.

Base case ($l = 0$): $a^{[0]}(x) = x$ and $a^{[0]}(x') = x'$, so $a_i^{[0]}(x') - a_i^{[0]}(x) = x'_i - x_i \in [I_i^{\mathrm{dn}[0]}, I_i^{\mathrm{up}[0]}]$ by assumption.

Inductive step — linear layer. Assume $\Delta_i := a_i^{[l-1]}(x') - a_i^{[l-1]}(x) \in [I_i^{\mathrm{dn}[l-1]}, I_i^{\mathrm{up}[l-1]}]$ for all i. The pre-activation difference at neuron j is $z_j^{[l]}(x') - z_j^{[l]}(x) = \sum_i W_{ji}^{[l]} \Delta_i$. Decompose $W_{ji}^{[l]} = W_{ji}^{+} + W_{ji}^{-}$ with $W_{ji}^{+} \geq 0$, $W_{ji}^{-} \leq 0$. Then:

$$z_j^{[l]}(x') - z_j^{[l]}(x) \;\leq\; \sum_i W_{ji}^{+}\, I_i^{\mathrm{up}[l-1]} + \sum_i W_{ji}^{-}\, I_i^{\mathrm{dn}[l-1]} \;=\; I_j^{\mathrm{up}[l]},$$
$$z_j^{[l]}(x') - z_j^{[l]}(x) \;\geq\; \sum_i W_{ji}^{+}\, I_i^{\mathrm{dn}[l-1]} + \sum_i W_{ji}^{-}\, I_i^{\mathrm{up}[l-1]} \;=\; I_j^{\mathrm{dn}[l]},$$

using monotonicity of multiplication by a non-negative (resp. non-positive) scalar. The bias cancels in the difference.

Inductive step — ReLU layer. Write $\Delta_j^z := z_j^{[l]}(x') - z_j^{[l]}(x) \in [I_j^{\mathrm{dn}[l]}, I_j^{\mathrm{up}[l]}]$. For any $u, v \in \mathbb{R}$: $\max(0,u) - \max(0,v) \leq \max(0, u-v)$ and $\max(0,u) - \max(0,v) \geq \min(0, u-v)$. Applying with $u = z_j(x')$, $v = z_j(x)$:

$$\min(0, \Delta_j^z) \;\leq\; a_j^{[l]}(x') - a_j^{[l]}(x) \;\leq\; \max(0, \Delta_j^z).$$

Since $\Delta_j^z \leq I_j^{\mathrm{up}[l]}$, we have $\max(0, \Delta_j^z) \leq \max(0, I_j^{\mathrm{up}[l]})$, and since $\Delta_j^z \geq I_j^{\mathrm{dn}[l]}$, we have $\min(0, \Delta_j^z) \geq \min(0, I_j^{\mathrm{dn}[l]})$. The updated interval $[\min(0, I_j^{\mathrm{dn}[l]}),\ \max(0, I_j^{\mathrm{up}[l]})]$ is exactly what the ReLU propagation step produces.

Corollary 3 (Constraint validity). *Every feasible point $(x, x', \{a^{\langle m \rangle [l]}\})$ of the MIP already satisfies constraints* (9)*. Adding them therefore does not cut off any feasible solution, but shrinks the LP relaxation polytope, strengthening the dual bound and reducing branch-and-bound effort.*

8.3 Inter-Network Interval Bounds

Our third technique exploits the structural similarity between N_1 and N_2. When N_2 is obtained by fine-tuning or distilling N_1, their weights are close, and we can bound the inter-network activation differences.

We compute interval bounds on the activation difference $a^{N_2[l]} - a^{N_1[l]}$ at each layer l, using the weight-difference matrices:

$$\Delta W^{[l]} = W_2^{[l]} - W_1^{[l]}, \qquad \Delta b^{[l]} = b_2^{[l]} - b_1^{[l]}.$$

Let $[\alpha^{[l]}, \beta^{[l]}]$ bound $a^{N_2[l]} - a^{N_1[l]}$, and let $\mathbf{u}^{[l-1]}$ be the upper bound on $a^{N_1[l-1]}$ from standard interval arithmetic. The propagation recursion is:

$$\beta^{[l]} = (W_1^{[l]})^{+}\beta^{[l-1]} + (W_1^{[l]})^{-}\alpha^{[l-1]} + |\Delta W^{[l]}|\,\mathbf{u}^{[l-1]} + \Delta b^{[l]}, \tag{10}$$
$$\alpha^{[l]} = (W_1^{[l]})^{+}\alpha^{[l-1]} + (W_1^{[l]})^{-}\beta^{[l-1]} - |\Delta W^{[l]}|\,\mathbf{u}^{[l-1]} + \Delta b^{[l]}. \tag{11}$$

The resulting MIP constraints are:

$$a^{N_1}{}_j^{[l]} + \alpha_j^{[l]} \;\le\; a^{N_2}{}_j^{[l]} \;\le\; a^{N_1}{}_j^{[l]} + \beta_j^{[l]}. \tag{12}$$

These constraints are particularly effective when N_1 and N_2 are structurally similar, as the weight differences are small and the resulting intervals are tight.

8.4 Dependency Analysis

Our fourth technique identifies *monotone relationships* between the activations of the clean and perturbed forward passes. We track the direction in which each activation responds to the perturbation using a *dependency signature* $\varphi_j \in \{-1, 0, +1, \text{NaN}\}$: $+1$ means the activation is monotonically increasing with the perturbation, -1 means decreasing, 0 means unchanged, and NaN means the direction is ambiguous.

Initialization (input layer). The signature depends on the perturbation type:

- ℓ_∞: $\varphi_i^{[0]} = \text{NaN}$ for all i (each pixel can increase or decrease).
- Brightness: $\varphi_i^{[0]} = +1$ for all i (pixels can only increase).
- Contrast: $\varphi_i^{[0]} = +1$ for all i.
- Occlusion: $\varphi_i^{[0]} = -1$ for $i \in P$, else 0 (occluded pixels decrease, others unchanged).
- Patch: $\varphi_i^{[0]} = \text{NaN}$ for $i \in P$, else 0.

Propagation through a linear layer. Each output neuron j merges the signatures of its inputs, weighted by the sign of the connecting weight:

$$\varphi_j^{[l]} \;=\; \bigoplus_{i:\, W_{ji}^{[l]} \neq 0} \operatorname{sign}\!\big(W_{ji}^{[l]}\big) \cdot \varphi_i^{[l-1]},$$

where $\oplus$ returns the common value if all operands agree, otherwise NaN.

Propagation through ReLU. If the neuron is always active ($z_j \ge 0$), the signature is inherited. If the neuron is always inactive ($z_j \le 0$), the signature is 0. Otherwise, the signature is NaN.

Added MIP constraints. For each neuron where the signature is determined:

$$\varphi_j^{[l]} = +1 \;\Rightarrow\; a^{\langle pert\rangle}{}_j^{[l]} \;\ge\; a^{\langle org\rangle}{}_j^{[l]}, \tag{13}$$

$$\varphi_j^{[l]} = -1 \;\Rightarrow\; a^{\langle pert\rangle}{}_j^{[l]} \;\le\; a^{\langle org\rangle}{}_j^{[l]}, \tag{14}$$

$$\varphi_j^{[l]} = 0 \;\Rightarrow\; a^{\langle pert\rangle}{}_j^{[l]} \;=\; a^{\langle org\rangle}{}_j^{[l]}. \tag{15}$$

These ordering constraints prune the search space by eliminating variable assignments that violate the proven monotone relationships.

9 Complete MIP Formulation

In this section, we present the complete transfer-mode MIP with all four tightening techniques enabled. The formulation brings together the transfer constraints from Section 5 and the tightening constraints from Section 8 into a single optimization problem.

The objective is:

$$\max_{x,\, x',\, \{a^{\langle m\rangle[l]}\},\, \{b^{\langle m\rangle[l]}\}} \mathcal{C}(N_2, x, c) - \mathcal{C}(N_1, x, c) \tag{16}$$

subject to the following constraints:

(Perturbation)	$x' \in \mathcal{I}_\varepsilon(x)$	(17)
(T1)	$\mathcal{C}(N_1, x, c) \geq \delta_1 + 10^{-3}$	(18)
(T2)	$\mathcal{C}(N_2, x, c) \geq \mathcal{C}(N_1, x, c)$	(19)
(T3)	Targeted or untargeted attack (Section 6)	(20)
(ReLU encoding)	Big-M for each network pass: $N_1(x)$, $N_2(x)$, $N_2(x')$	(21)
(Pert. diff. intervals)	$a^{\langle org\rangle[l]}_j + I^{\mathrm{dn}[l]}_j \leq a^{\langle pert\rangle[l]}_j \leq a^{\langle org\rangle[l]}_j + I^{\mathrm{up}[l]}_j$	(22)
(Inter-network intervals)	$a^{N_1[l]}_j + \alpha^{[l]}_j \leq a^{N_2[l]}_j \leq a^{N_1[l]}_j + \beta^{[l]}_j$	(23)
(Dependency constraints)	$a^{\langle pert\rangle[l]}_j \gtrless a^{\langle org\rangle[l]}_j$ per sign of $\varphi^{[l]}_j$	(24)
(Warm-start)	$b^{\mathrm{hint}}_j \leftarrow$ PGD activation signs	(25)

We note the following about the formulation:

- The MIP encodes three forward passes: $N_1(x)$, $N_2(x)$, and $N_2(x')$. The pass $N_1(x')$ is not needed since no constraint involves N_1's output on the perturbed input.
- Constraints T1–T3 are encoded via the big-M confidence-margin linearization described in Section 4.
- The perturbation-difference interval constraints (6th row) are added by the second tightening technique (Section 8.2).
- The inter-network interval constraints (7th row) are added by the third tightening technique (Section 8.3).
- The dependency constraints (8th row) are added by the fourth tightening technique (Section 8.4).
- The binary warm-start (9th row) is provided by the hyper-adversarial attack (Section 8.1).

10 Inference-Time Decision Procedure

In this section, we describe how the precomputed transfer guarantee is applied at inference time. Given the standard-mode value δ_1 (from Section 4) and the

transfer value δ_{diff} (from Section 5), the runtime procedure certifies individual inputs using only two forward passes.

For each input x, the procedure applies a two-stage test:

1. **Standard-mode check.** Compute $c_1 := \mathcal{C}(N_1, x, c)$. If $c_1 < \delta_1$, the input cannot be certified as robust for N_1 (or, consequently, for N_2).
2. **Transfer check.** Compute $c_2 := \mathcal{C}(N_2, x, c)$. If $c_2 - c_1 > \delta_{\text{diff}}$, certify x as $(\varepsilon, c \to t)$-robust for N_2.

By Corollary 1, an equivalent single-threshold condition is:

$$c_1 \geq \delta_1 \text{ and } c_2 > \delta_1 + \delta_{\text{diff}} \implies x \text{ is } (\varepsilon, c \to t)\text{-robust for } N_2.$$

This procedure requires only two forward passes at inference time and no additional optimization. The MIP is solved once offline, and the resulting thresholds δ_1 and δ_{diff} are used for all subsequent inputs.

Note that, in the notation of Section 2.5, $\delta_1 = \delta_1(N_1)$ and $\delta_{\text{diff}} = \delta^*_{\text{diff}}$. By Theorem 2, $\delta_1(N_2) \geq \delta_1(N_1) + \delta^*_{\text{diff}}$, so the threshold $\delta_1 + \delta_{\text{diff}}$ is always a valid (conservative) certificate for N_2. The certificate is strict: there may exist inputs with $\mathcal{C}(N_2, x, c) \in [\delta_1(N_1) + \delta^*_{\text{diff}},\ \delta_1(N_2)]$ that are also robust but not certified by this procedure. This gap is precisely characterized in Section 7.3.

11 Conclusion

In this work, we addressed the problem of verifying the transfer of robustness guarantees between neural networks. We extended VHAGaR with a transfer mode that encodes the problem as a mixed-integer program, simultaneously modeling three forward passes — $N_1(x)$, $N_2(x)$, and $N_2(x')$ — to capture the interaction between two networks under adversarial perturbation. Our approach supports the L_∞ perturbation and six semantic feature perturbations, and employs four complementary tightening techniques to make the MIP tractable.

We established three formal results about the transfer MIP:

1. **Theorem 2**: the original transfer MIP provides a sound lower bound, $\delta_1(N_2) \geq \delta_1(N_1) + \delta^*_{\text{diff}}$. This enables certifying N_2's robustness by leveraging N_1's precomputed guarantee.
2. **Counterexample** (Section 7.2): the reverse inequality $\delta_1(N_1) + \delta^*_{\text{diff}} \geq \delta_1(N_2)$ is false in general. The gap equals the surplus N_1-confidence at the transfer optimum ($\mathcal{C}(N_1, x^*) - \delta_1(N_1)$), which is always positive.
3. **Theorem 3**: the modified transfer MIP provides a matching upper bound, $\delta_1(N_1) + \hat{\delta}_{\text{diff}} \geq \hat{\delta}_1(N_2)$, over the joint-foolable region $\mathcal{F}_{1\wedge 2} = \mathcal{F}_1 \cap \mathcal{F}_2$.

Together, the two formulations provide a two-sided characterization of the robustness relationship between any pair of networks: the original MIP establishes how much robustness is guaranteed to transfer, while the modified MIP bounds how much can be gained in the shared vulnerability region.

Table 5. Summary of the four tightening techniques.

Technique	Core method	Effect on MIP
Hyper-adversarial attack	PGD attack $\to v^\star$	Solver incumbent; binary hints b_j
Pert.-diff. interval propagation	$[I^{\mathrm{dn}[l]}, I^{\mathrm{up}[l]}]$ propagation	Bounds $a^{\langle pert \rangle [l]} - a^{\langle org \rangle [l]}$ per layer
Inter-network intervals	$[\alpha^{[l]}, \beta^{[l]}]$ via ΔW	Bounds $a^{N_2[l]} - a^{N_1[l]}$ per layer
Dependency analysis	$\varphi^{[l]} \in \{-1, 0, +1, \mathrm{NaN}\}$	Monotone equality/inequality constraints

11.1 Summary of Tightening Techniques

Table 5 summarizes the four tightening techniques and their effects on the MIP.

11.2 Future Work

Several natural extensions of this work remain. First, generalizing the transfer formulation from two classes to the full K-class setting would broaden its applicability. Second, exploring non-ℓ_∞ perturbation sets beyond those already supported could address a wider range of adversarial threat models. Third, developing analogous bounds for backbone-sharing architectures, where N_1 and N_2 share a common encoder, could exploit additional structural constraints. Finally, closing the gap between $\delta_1(N_1) + \delta^*_{\mathrm{diff}}$ and $\delta_1(N_2)$ by searching over the full $\mathcal{F}_2$ region with additional structural constraints is an important direction for obtaining tighter transfer guarantees.

References

1. Katz, G., Barrett, C., Dill, D.L., Julian, K., Kochenderfer, M.J.: Reluplex: An efficient SMT solver for verifying deep neural networks. In: CAV (2017)
2. Elboher, E., Günther, J., Katz, G.: An Abstraction-Based Framework for Neural Network Verification. In: Computer Aided Verification (CAV) (2020)
3. Anderson, R., Huchette, J., Ma, W., Tjeng, V., Vielma, J.P.: Strong Mixed-Integer Programming Formulations for Trained Neural Networks. Mathematical Programming **183**(1), 3–39 (2020)
4. Gurobi Optimization, LLC: Gurobi Optimizer Reference Manual (2023)
5. Dunning, I., Huchette, J., Lubin, M.: JuMP: A Modeling Language for Mathematical Optimization. SIAM Review **59**(2), 295–320 (2017)
6. Singh, G., Gehr, T., Püschel, M., Vechev, M.: An abstract domain for certifying neural networks. In: POPL (2019)
7. Wang, S., Zhang, H., Xu, K., Lin, X., Jana, S., Hsieh, C.-J., Kolter, J.Z.: Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In: NeurIPS (2021)
8. Tjeng, V., Xiao, K., Tedrake, R.: Evaluating robustness of neural networks: An extreme value theory approach. In: ICLR (2019)

9. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. In: ICLR (2018)
10. Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., Fergus, R.: Intriguing properties of neural networks. In: ICLR (2014)
11. Carlini, N., Wagner, D.: Towards evaluating the robustness of neural networks. In: IEEE S&P (2017)